Arrays

"Array" is a C++ variable type analogous to vectors and matrices

Important applications:

a) storing and manipulating of vectors / matrices / data

b) calculating statistics (mean, standard deviation, median, etc.)

c) strings / text (arrays of characters)

d) processing experimental data, plotting, graphics, audio, etc.

e) discrete approximation of functions (computer solution of calculus problems, etc.)

1D Arrays

analogous to vectors

all basic C++ variable types (int, double, char, etc.) can be declared as 1D arrays

syntax: (variable type) (array name) [size] , eg

```
int A[3];
A[0] = 1;
A[2] = 3;
```

array indices range from 0 to size-1 (0 to 2 for the example)

size must be a constant in C++ (variable size is not allowed)

Initialization of 1D Arrays

a) Initialization during declaration, eg

```
double B[3] = { 1.1, 2.2, 3.3 };
```

this type of expression can only be used during declaration

b) During declaration with auto computation of size, eg

```
double B[] = { 1.1, 2.2, 3.3 };
```

note you should be careful when using []
-- there are only two situations you can use [] including this one

c) During declaration to all zero elements, eg

```
double B[3] = { 0.0 };
```

- note C++ doesn't initialize arrays to zero by default -- you have to specify it like this if you want

d) Using a for loop and expression for each element, eg

```
double B[3];
for(int i = 0; i < 3; i++) B[i] = 0.1;
```

e) Using a for loop and keyboard input for each element, eg

```
double B[3];
for(int i = 0; i < 3; i++) cin >> B[i];
```

f) Using a for loop and file input for each element, eg

```
double B[3];
ifstream fin("file.txt"); // assume "file.txt" exists with 3 elements
for(int i = 0; i < 3; i++) fin >> B[i];
```

Typical Errors

a) int n = 3;
   int A[n];

array size has to be constant

b) int A[];

array size has to be a constant [] means nothing

c) int A[100000000];

a standard array can't get much larger than 10000
(the max size depends on the compiler)

d) int A[3]; // where A represents a real vector

for real number elements we need double / float

e) int A[3];
   cout << A[1];

uninitialized array elements are garbage

f) int A[3] = { 1, 2, 3 }, B[3];
B = A;

you can't put an array on the LHS of an an assignment
you can only put array elements on the LHS, eg A[2] = 1

g) int A[3], i = 3, k;
cout << A[i];
A[-1] = 1;
A[5] = 3;
A[k] = 5;

A is out of range -- output of range output is garbage
note: if i = 2 then A[i] is uninitialized garbage

out of range A goes from A[0] to A[2]
-- this could crash the program since you
are putting 1 into some unknown memory

k is garbage -- potentially out of range

h) int A[3];
A = { 1, 2, 3 };

outside of an array declaration you can only set arrays one element at a time (eg A[i] = 3)

same error as f) but also can't set array to a list
outside of a declaration

2D Arrays

analogous to matrices

all basic C++ variable types (int, double, char, etc.) can be declared as 2D arrays

syntax: (variable type) (array name) [row_size][col_size] , eg

int A[2][3]; // declare a 2D array with 2 rows and 3 columns
A[0][0] = 1;
A[1][2] = 3;

array indices range from 0 to size-1 ([0][0] to [1][2] for example)

size must be a constant in C++ (variable size is not allowed)

Initialization of 2D Arrays

a) Initialization during declaration, eg

row1 , row2 - initialize one row at a time

```
double B[2][3] = { 1, 2, 3, 4, 5, 6 };
double B[2][3] = { { 1, 2, 3 }, { 4, 5, 6 } };
```

elements = n * m

b) During declaration with auto computation of row size, eg

```
double B[][3] = { 1, 2, 3, 4, 5, 6 }; // col size must always be given
```

note you should be careful when using []
-- there are only two situations you can use [] including this one

c) During declaration to all zero elements, eg

```
double B[2][3] = { 0.0 };
```

d) Using a for loop and expression for each element, eg

```
double B[2][3];
for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 3; j++) {
        B[i][j] = 0.1;
    }
}
```

- initialize row by row -- row0, row1, ...

i=0,j=0,j=1,j=2 -> row0
i=1,j=0,j=1,j=1 -> row1

this for i, for j loop accesses
the 2D array one row at a time

e) Using a for loop and keyboard input for each element, eg

```
double B[2][3];
for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 3; j++) {
        cin >> B[i][j];
    }
}
```

f) Using a for loop and file input for each element, eg

```
double B[2][3];
ifstream fin("file.txt"); // assume "file.txt" exists with 6 elements

for(int i = 0; i < 2; i++) {
    for(int j = 0; j < 3; j++) {
        fin >> B[i][j];
    }
}
```

- read one row at a time from the file

- this is a nice general / practical way of initialization that doesn't require much typing and is good for experimental data files, tables, spreadsheets, etc.

Typical Errors

a) int A[2][3];
   cout << A[1][1];

- do this as an exercise problem (question #1) -- these errors are analogous to the typical errors for 1D arrays

b) int A[100000000][3], B[2][100000000];

c) int A[2][3]; // where A represents a real matrix

d) int n = 2, m = 3;
   int A[n][m];

e) int A[][], B[3][], C[][3];

f) int A[2][3], i = 1, k;
cout << A[i+1][i+1];
A[-1][i] = 1;
A[i][3] = 3;
A[1][k] = 7;

g) int A[2][3];
A = { 1, 2, 3, 4, 5, 6 };

h) int A[2][3] = { 1, 2, 3, 4, 5, 6 }, B[2][3];
B = A;

outside of an array declaration you can only set arrays one element at a time (eg A[i][j] = 3)